

Concurrency

Concurrency means doing more than one task during the same time period, instead of finishing one completely before starting the next. In operating systems, this happens when multiple processes or threads are active together and the CPU quickly switches between them so it feels like they are running at once.

Simple Real-World Example

Imagine a single waiter in a restaurant. He cannot serve one customer from start to end and then go to the next — instead:

- He takes an order from customer A
- While food is being prepared, he serves water to customer B
- Then he returns to customer A with the food
- Later he takes payment from customer C, and so on...

Even though he is just one person, multiple tasks progress together. None of them wait to finish fully before another begins. This is concurrency.

OS Perspective

Similarly, in a computer:

- One thread might be playing music
- Another is downloading a file
- Another opens a browser window

The processor keeps switching rapidly between them, so all seem to run at the same time. This smooth sharing of time and resources is concurrency.

2. Thread

A thread is like a small unit of execution inside a process.

- It is a **single flow of instructions** running inside a program.
- It creates an **independent execution path** inside the same process.
- It is called a **light-weight process** because it needs fewer resources than a full process.
- A big task in a program can be split into **multiple threads** so they can run in parallel.

Real-world example:

When you open a browser with multiple tabs — each tab is a thread.

3. Thread Scheduling

All threads don't run at the exact same instant.

The OS gives each thread small time slices to run, based on its priority.

That's how multiple threads appear to work together without waiting for each other.

4. Thread Context Switching

When the CPU switches from one thread to another, it saves the current thread's information and loads the next one.

- OS stores the thread's registers, program counter, and stack.
- It **does not switch the memory space** because threads of the same process share it.
- This switching is **faster than process switching**.
- CPU cache remains intact, so execution resumes quickly.

5. How does every thread get CPU access?

- Every thread has its **own Program Counter (PC)**.
- Scheduler decides which thread will run next based on scheduling policies.
- CPU fetches instructions pointed by the PC of the selected thread and executes them.

6. I/O or Time Quantum-based Thread Switching

Context switching occurs when:

- A thread is waiting for I/O, or
- Its time slice completes.

Just like processes have **PCB**, threads have **TCB (Thread Control Block)** which stores thread-specific information during switching.

7. Does Multithreading Help on a Single CPU?

- In a **single CPU system**, multithreading does **not give performance gain**.
- Because both threads need the same CPU and keep switching between each other.
- There is no real parallelism — only rapid switching.

8. Benefits of Multithreading

- **Responsiveness:** Program doesn't freeze even when one task takes time.
- **Resource Sharing:** Threads share memory and data easily within the same process.
- **Economical:** Creating threads costs less than creating multiple processes.
- **Better Utilization of Multi-Core CPUs:** On multi-core systems, threads can truly run in parallel.